SPHERE WEB AUTOMATION

Course Completion Script Step-by-Step Walkthrough

How `tests/CourseComplete.spec.ts` drives a whole course to completion — the sidebar loop that decides what to open next, the three-tier answer strategy behind its assessments, and the single assertion that decides whether any of it worked.

Script tests/CourseComplete.spec.ts · 3,195 lines · **1 test**, 46 helpers, **1 expect()**

Target any course on the target environment (login + COURSE_NAME required)

Run npm run run:course

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	The shape of the file	Diagram — seven subsystems, 46 helpers
3	The eight steps	Diagram — the orchestrator, end to end
4	Configuration and inputs	Seven constants, three env vars
5	The sidebar-driven loop	How it picks what to open next
6	Module detection and the handlers	Video · PDF · content · assessment
7	Answering an assessment	Diagram — the three-tier strategy ladder
8	The answer-key plumbing	Interceptor, artifact fetch, poison rule
9	The Gemini fallback	Batching, quota, caching, model fallback
10	Completion, certificate, report	Five signals and what follows them
11	What green actually proves	One assertion, and what it does not cover
12	Artifacts, run variants, failure modes	Troubleshooting reference

How to read this document

This is a different kind of script from the others in the series. Those are element sweeps: many small tests, each asserting one thing. This one is a **robot learner** — a single test that opens a course and works through it until the platform says it is finished. Almost all of its 3,195 lines are the machinery that makes that possible.

So the structure follows the machinery rather than a list of tests. Section 3 diagrams the eight steps the one test takes; Sections 5 to 10 open up the interesting ones. Section 11 is the honest accounting: what a green run does and does not prove.

> **Line numbers in this document are live**
> Every reference points at `tests/CourseComplete.spec.ts` as it stands today — 3,195 lines, including the per-line explanatory comments carried across the suite. They are safe to click through to.

> **A note on scale**
> At 3,195 lines this is the largest file in the suite by a wide margin — roughly six times the size of the Course TOC spec, for a twentieth of the test count. That ratio *is* the story: driving a course to completion means handling every module type, every assessment shape, and every way the platform can stall, and none of that can be expressed as an assertion.

1 What this script does

It behaves like a learner who is determined to finish. It searches for a course, opens it, and then works through every module — watching videos, paging PDFs, scrolling content, and *answering assessments correctly* — until the platform shows the Congratulations screen.

Everything else in the suite checks that a page renders. This one checks that **the product actually works end to end**, which is a different and much harder question. A course only marks a module complete when the platform's own progress calls say so, and a graded assessment only completes when the pass mark is met. There is no way to fake either.

The four things that make it hard

Modules are gated

- Later modules stay locked until earlier ones register as complete
- So the order of work is decided by the platform, not the script
- Clicking "Next" repeatedly just loops through the same units

Assessments must be passed

- Answering "A" every time fails the pass mark
- A failed assessment never marks its module complete
- So the script has to know the right answers

Nothing is uniform

- Four module types, detected rather than declared
- Radio and checkbox questions, mixed within one quiz
- Option order is shuffled on every render

Where it gets the answers

This is the part of the script people ask about first. Sphere prefetches a JSON file for each assessment, and that file carries an `isCorrect` flag on every option. The script listens for those responses and files them by resource id. Where the copy the browser receives has had its flags stripped — which graded assessments do deliberately — it goes to the **published artifact on S3**, which still carries them. Only when neither source yields a usable key does it fall back to asking Gemini.

Section 7 diagrams that ladder and Section 8 covers the plumbing. The short version is that **the AI is the last resort, not the mechanism** — on a normal published course it is never called at all.

> **This is a test account doing coursework, not a person**
> Worth being explicit, because the script does real things to real data: it enrols, completes modules, submits assessment answers and generates a certificate on whichever account it is pointed at. It is meant for a test learner on a test environment. The one place it deliberately holds back is the ratings popup — `handleCompletionPopups()` closes it with the X rather than submitting a rating, so a run never leaves a fabricated review on a real course.

Entry points

Command	What it runs
`npm run run:course`	The interactive runner (`scripts/run-course.mjs`) — prompts for the account and the course, then launches this spec. The normal way in.
`npm run courses:list`	Lists the courses the runner knows about, so you can pick a name that will actually match.
`npm run run:course:env`	Runs it directly against staging using whatever `COURSE_NAME` is already set.
`npm run run:course-fresh`	Mints a brand-new account first (`create-account` , Step 5), then completes the course as that user. The only way to exercise a *first* completion rather than a re-run.

Command	What it runs
`npm run report:course`	Headless, pinned to one course, writing a blob report. What CI uses.

```
# PowerShell — the direct form from the file header
$env:COURSE_NAME="Normal Labour & Birth and AMTSL"
$env:GEMINI_API_KEY="…"            # optional — only used if a key is unusable
npx playwright test tests/CourseComplete.spec.ts --headed
```

One hour, one test

`test.setTimeout(60 * 60 * 1000)` — a full course with several assessments genuinely takes tens of minutes, and the script would rather run long than abandon a course halfway. Section 12 covers what to look at when a run hits that ceiling.

2 The shape of the file

One test, 46 helpers, seven subsystems. The test itself is only the last 390 lines; everything before it is the machinery it calls.

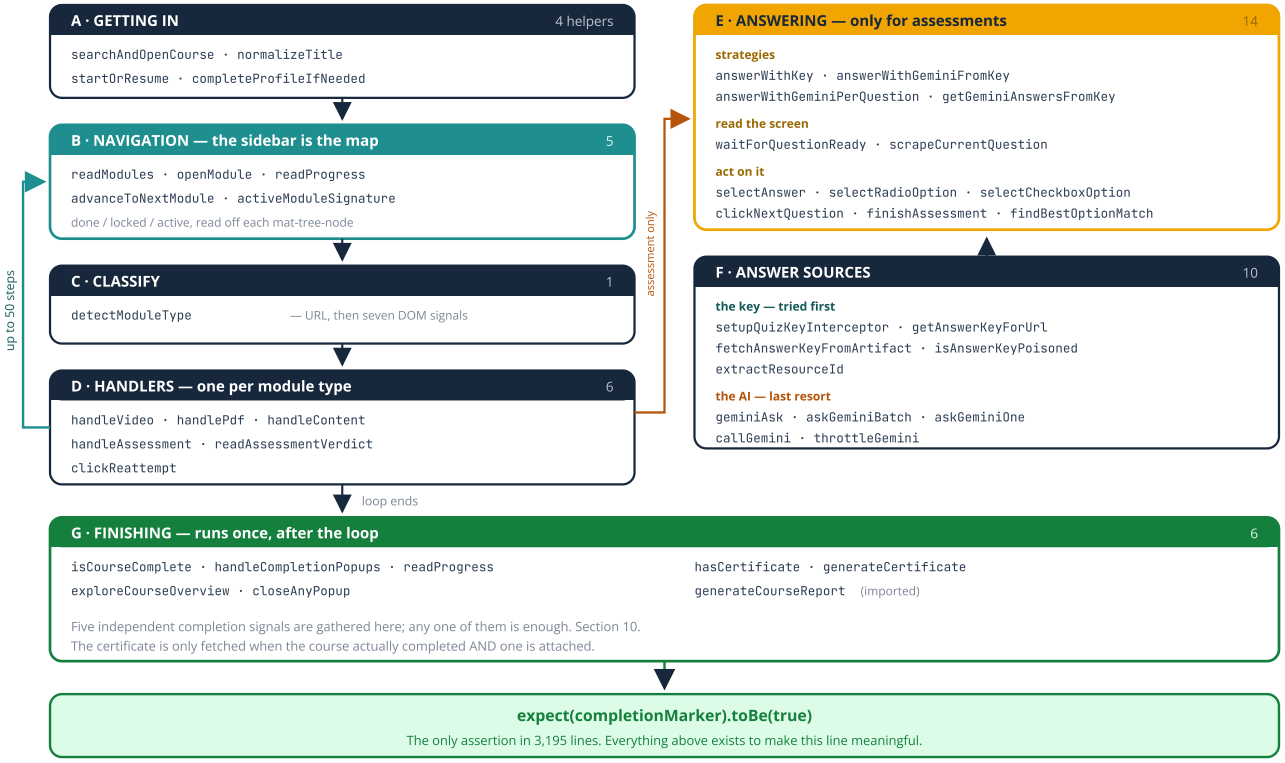


Figure 1 — the seven subsystems, and how the single test threads through them. The loop on the left runs once per module; the gold box on the right runs only when that module is an assessment.

3 The eight steps

The test body is a numbered sequence, and the source labels it that way. Seven of the eight steps are short. Step 5 is the script.

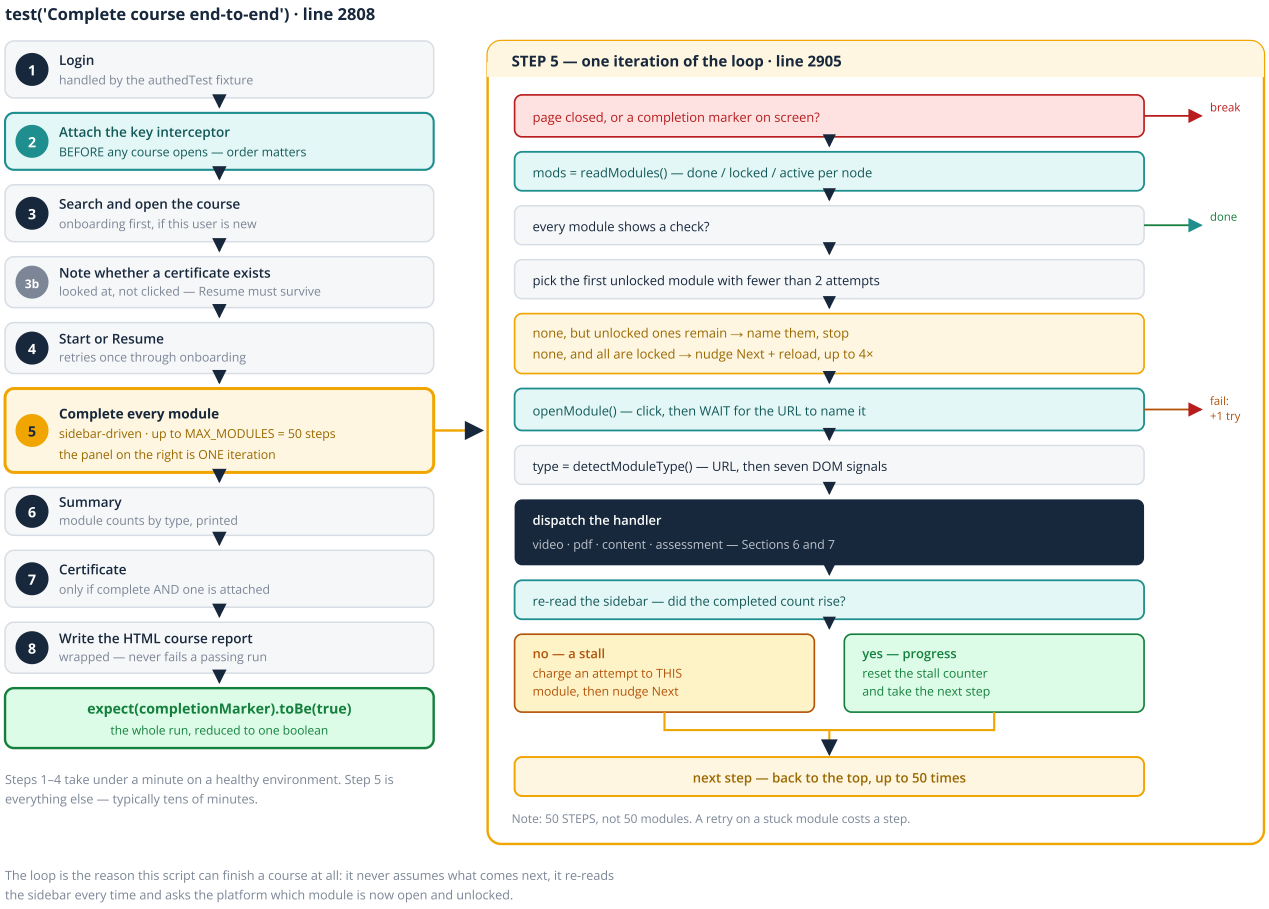


Figure 2 — the eight steps of the single test, with one iteration of the module loop expanded. Red exits end the loop; amber ones cost the current module an attempt.

4 Configuration and inputs

Seven tuning constants and three environment variables. Two of the three inputs are optional; the third decides whether the test runs at all.

Inputs

Variable	Required	What it does
`COURSE_NAME` line 87	yes	Which course to complete. There is no default, and unlike the Course TOC spec this one **throws rather than skips** — the error names both ways to set it, including the interactive runner.
`GEMINI_API_KEY` line 89	optional	Enables the AI fallback. Without it, an assessment whose key is missing or poisoned cannot be answered reliably. The run prints `⚠️ No GEMINI_API_KEY` at the top so this is visible before the failure rather than after.
`BASE_URL` line 43	defaulted	Which environment. Defaults to staging.
`USE_FRESH_USER`	optional	Reuse the session of an account just minted by the create-account suite, so the run exercises a genuine *first* completion rather than re-walking a finished course.
`VERBOSE_CHECKBOX`	optional	Logs every strategy the checkbox clicker tries. Only useful when a multi-select question will not tick.

It reads `.env` itself, without a library

Lines 66–84 are a small immediately-invoked function that parses `.env` by hand. Two details are deliberate: it splits on the **first** `=` only, so a value containing `=` (an API key, typically) survives intact; and it **never overwrites a variable already set in the real environment**, so a value passed on the command line always beats the file. Writing it out rather than importing `dotenv` keeps the spec runnable on its own.

The seven constants

Constant	Value	What it governs
`MAX_MODULES` line 51	**50**	The ceiling on loop *steps*. Not a module count — a retry on a stuck module spends one. It exists so a course that never registers progress ends rather than running for an hour.
`VIDEO_WATCH_MS` line 52	**5 s**	The settle after a video finishes, giving the platform's progress call time to land. Also the fallback wait when no `<video>` element ever appears.
`PDF_MAX_PAGES` line 53	**60**	How many times the PDF viewer's own chevron is clicked before giving up. The outer Next button stays disabled until every page has been turned.
`ASSESSMENT_MAX_RETRIES` line 54	**2**	Re-attempts when the results screen says the pass mark was missed. Without this, a failed graded assessment would leave its module permanently incomplete.
`GEMINI_RPM_DELAY` line 55	**6.5 s**	Self-throttle between AI calls, sized for the free tier's 10-requests-per-minute limit.
`GEMINI_RETRY_WAIT_MS` line 56	**20 s**	How long to wait out a rate-limit response that does not say how long to wait.
`GEMINI_MAX_RETRY_WAIT_MS` line 57	**65 s**	A ceiling on any single rate-limit wait, so a server-suggested delay of ten minutes cannot stall the run.

Two model names sit alongside them: `gemini-2.5-flash` as the primary and `gemini-2.5-flash-lite` as the higher-quota fallback (lines 60 and 62). Section 9 covers when the script moves between them.

> **None of the seven read the environment**
> Unlike the Course TOC spec, where every timeout is overridable, these are compiled-in. In practice that has been fine — they are tuned to platform behaviour rather than to machine speed — but it does mean that adapting to a slower environment means editing the file. The four Gemini values are the ones most likely to need it, since they encode a specific vendor's free-tier limits.

5 The sidebar-driven loop

The single most important design decision in the file, and the source says why in one line: *"Blindly clicking Next just loops through the units."*

A course player has a Next button, and the obvious way to complete a course is to keep pressing it. That does not work here. Sphere's player advances within a unit, so pressing Next walks the same handful of screens forever while the module list stays untouched. Worse, it gives no way to know when you are done.

So the script drives from the **sidebar** instead — the module tree that is the platform's own statement of what exists, what is finished, and what is still locked. Every iteration re-reads it, decides what to open, opens it, and then re-reads it again to ask whether anything actually changed.

What one sidebar node tells you

`readModules()` (line 2653) runs inside the browser and turns every `mat-tree-node` into a plain object. Four of its fields are read from markup that was never designed to be read this way:

Field	How it is derived	Why that way
`done`	`/check/.test(iconTxt) && !/outline/.test(iconTxt)`	Material icons render their name as ligature text, so the icon *is* readable — but `check_circle_outline` means "not done" and contains the word "check". **Excluding the outline variant is the whole correctness of the completion check.**
`locked`	`/\block\b/.test(iconTxt) || /\bdisabled\b/.test(cls)`	A lock icon or a disabled class. Either can carry the state depending on build.
`active`	`/content-active|li-active/`	Two class names, because the player has shipped both.
`resourceId`	`extractResourceId(href)`	The `do_…` id parsed out of the node's router link. This is the ground truth for "did we land on the module we clicked" — see below.

Note that the class list is built from **both** the node and its inner anchor (`` `${el.className} ${a?.className}` ``), because the state lives on one or the other depending on version. And the whole thing ends in `.catch(() ⇒ [])`: an unreadable sidebar yields an empty list, which the loop treats as "no sidebar" and falls back to processing whatever is on screen.

The bug that made `openModule()` necessary

Clicking a sidebar link is not enough. The player is a single-page app, so the previous viewer stays mounted for a beat after the click. The original code would click a quiz, immediately detect the still-showing video, **complete that video a second time, credit it to the quiz, and then blame the quiz for never registering**. The doc comment above it, at line 2699, records exactly this.

The fix is to treat the URL as the arbiter. `openModule()` reads the target `do_…` id from the node's href, clicks, then polls for up to six seconds for the URL to actually name that resource. If the click did not navigate it goes to the href directly, polls again, and only then gives up — returning `false` so the caller can charge an attempt rather than process the wrong module.

Three ways the loop can get stuck, and what it does about each

Situation	Response	Why not just stop
A module opens but never registers as complete	Charge an attempt to *that module by name*. After 2 attempts the picker skips it and moves to the next unfinished one.	A single stuck module used to end the whole run. Now the run continues and only stops once **every** remaining module is exhausted — so one broken video no longer hides the state of the other fifteen.
Everything left is locked	Nudge Next, reload if the page drifted off the overview, re-read the sidebar. Up to 4 times.	The final gated unit often unlocks a beat after its prerequisites register, or needs a Next nudge to enter. Conceding immediately would fail a course that was about to finish.
No sidebar at all	Process whatever is on screen; give up after 3 consecutive steps with no progress.	With no module list there is nothing to attribute a stall to, so the old global rule is the only one available.

Each of these writes a line into `reportWarnings` as well as the console, so the HTML report at the end names the modules that would not complete rather than just saying the course did not finish.

```
// line 2939 — when unlocked modules remain but all have hit the cap
console.log(`\n🛑 These modules would not register as complete after ${MODULE_MAX_ATTEMPTS} attempts
each:`);
unlocked.forEach((m) ⇒ console.log(`     • ${m.text}`));
```

Why the loop counts steps rather than modules

`for (let step = 1; step ⩽ MAX_MODULES; step++)` — the comment on line 2904 is explicit that these are steps, not modules. A retry spends one, a lock nudge spends one, an open failure spends one. Fifty is therefore comfortably more than any real course needs while still being a hard stop: the worst case is fifty iterations, not fifty modules each retried twice.

6 Module detection and the handlers

Sphere does not tell the script what kind of module it just opened. `detectModuleType()` works it out, and gets it wrong at some cost — a video handled as content is never watched, and never completes.

Why the URL is not enough

Three routes are unambiguous: `/viewer/pdf` means PDF, `/viewer/video` means video, `/viewer/html` means content. The problem is the fourth. **Sphere serves course intro pages from `/viewer/quiz/` too**, so the quiz route is the one route that has to be verified rather than trusted.

For that case the function does three things in order of confidence:

1. **Check the captured answer keys first.** If the interceptor filed a quiz JSON under this exact resource id, the module *is* an assessment — the platform fetched its questions. This is the strongest possible signal and it short-circuits everything below.
2. **Gather seven DOM signals in one round trip.** A "Start Now" button, the radio count, the checkbox count, a "N Questions" + "N Minutes" pair, a canvas count, a PDF iframe, and whether the only heading is "Introduction" or "Overview".
3. **Let content beat the route.** PDF markers win outright — a canvas or a PDF iframe on a quiz URL is treated as a PDF, and the run logs that it did so. An Intro heading with *no* quiz controls at all is treated as content. Any single quiz signal is enough to call it an assessment.

If none of that resolves — a quiz URL with no quiz indicators anywhere — it returns `content` and says so in the log. Content is the safe default: handling it just scrolls the page, which is harmless if the guess was wrong.

The four handlers

Handler	Line	What it does, and what it is really trying to trigger
`handleVideo`	520	Six stages, and every one of them exists to make the platform record a view. Wait for the `<video>` element (30 s) → wait for metadata so the duration is known, calling `.load()` if the browser is being lazy (60 s) → wait for enough buffer to play → **mute and play**, because muting is what makes browsers permit programmatic autoplay → **seek to half a second before the end** so the `ended` event still fires naturally → await that event, with a 15-second safety net. Returns whether the metadata ever loaded; a false becomes a report warning, not a failure.
`handlePdf`	619	Clicks the viewer's *own* `chevron_right` up to 60 times, stopping when it disappears or goes disabled. The outer module Next button stays disabled until every page has been turned, so this is not cosmetic — it is the completion condition. Clicks are `force`d because the viewer overlay intercepts them.
`handleContent`	642	Scrolls to the bottom (which is what registers the page as read), back to the top so inner controls are in view, then flips up to 30 inner page-turn chevrons if the content has them.
`handleAssessment`	669	The large one — Section 7.

The video handler is the clearest example of the script's whole approach

It would be far simpler to call `video.currentTime = video.duration` and move on. That does not work: the platform listens for real playback events, so the script has to mute the element (to get autoplay permission), start it, seek to just short of the end (so `ended` fires naturally rather than being skipped past), and then wait for the event. Every one of those steps is a workaround for how browsers and the platform actually behave, and none of it would be discoverable from the DOM alone.

7 Answering an assessment

A graded assessment only marks its module complete when the pass mark is met, so guessing is not an option. The script has three ways to answer, and it always takes the most accurate one available.

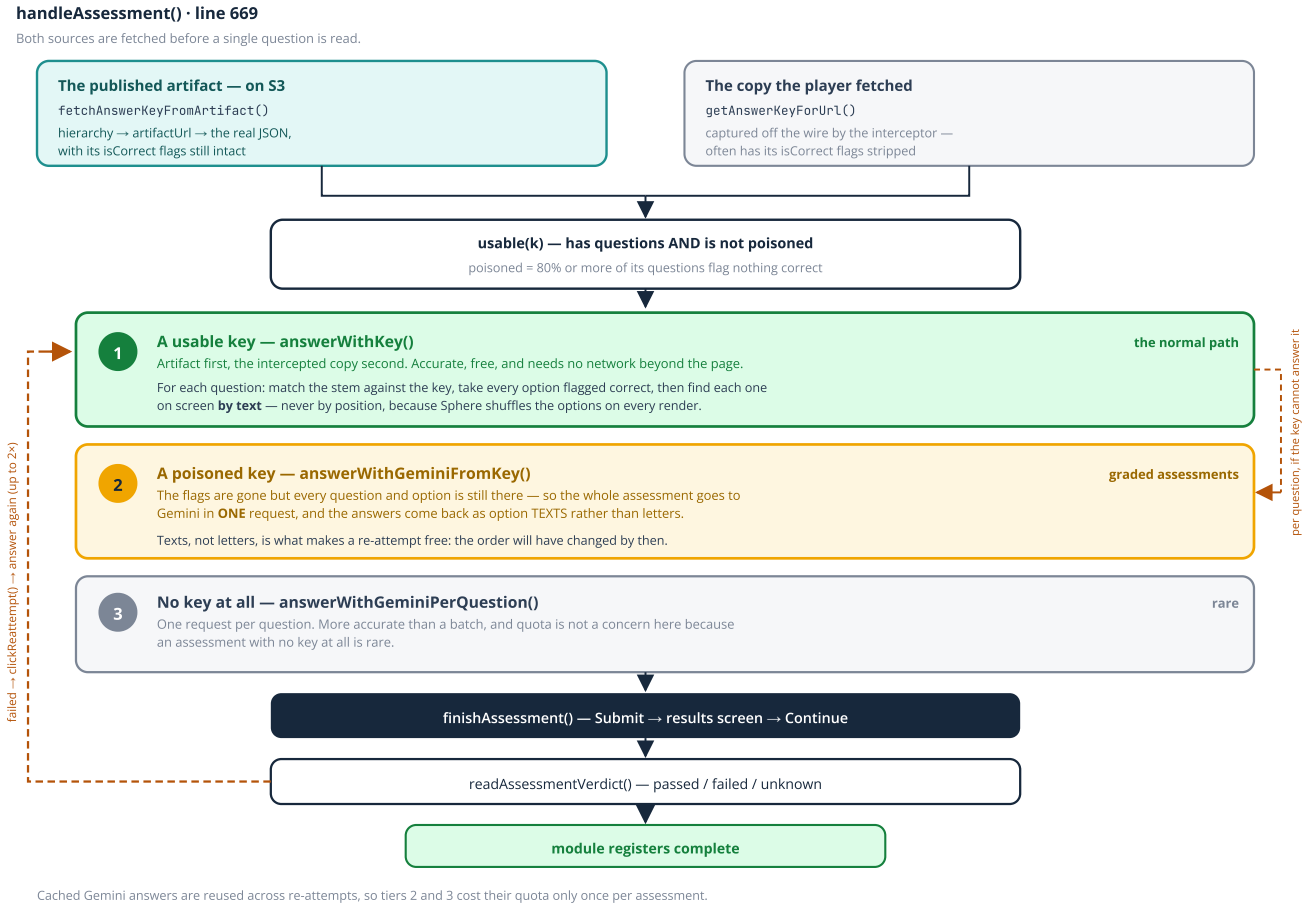


Figure 3 — the three-tier answer strategy. Tier 1 is the normal path on a published course; the AI is reached only when the key is missing or has been deliberately stripped.

7 Answering an assessment (continued)

The line that tells you which tier a run took

`handleAssessment()` announces its strategy before it reads a single question, and this is the most useful line in a run's log:

```
ASSESSMENT — using answer key from the published artifact for do_1134204940…
ASSESSMENT — every key for do_1134204940… is POISONED (all isCorrect: false) — using Gemini AI
ASSESSMENT — no answer key for do_1134204940…, will use Gemini AI
```

If every assessment in a run prints the first form, the AI was never called and the run cost nothing but time. If any prints the second or third, the run depended on `GEMINI_API_KEY`, and Section 9's quota limits apply.

Counting the questions is harder than it looks

The obvious source is the on-screen "3 / 12" counter. The source rejects it, and the reason is worth quoting: the regex `/\d+\/\d+/` **can match a value inside the question itself** — a blood-pressure reading of "100/70" makes the script believe there are 70 questions, and it then walks off the end of the assessment.

So the count comes from the key when there is one, *even a poisoned one* — a poisoned key has lost its answers but still knows exactly how many questions this assessment holds. The counter is only the fallback, and a hard-coded 5 the fallback's fallback:

```
const total = capturedKey && capturedKey.questions.length > 0
  ? capturedKey.questions.length
  : counterN || 5;
```

There is a safety net for the other direction too. If the answering loop reaches a question with no text *and* no options, it concludes the assessment really has ended and stops, rather than clicking Next into nothing.

When the key cannot answer one question

Three things can go wrong for a single question even with a good key: the matcher finds no entry for this stem, the matched entry flags nothing correct, or none of its correct options can be found on screen. The script's response to all three is the same, and it is a deliberate change from how it used to behave:

> **Ask the AI rather than defaulting to A**
> "A wrong answer costs the pass mark, and the module only completes when the assessment is passed." Falling back to option A is free but actively harmful — it turns a recoverable gap into a failed assessment and an incomplete module. So `answerWithKey()` carries a small `askGemini()` closure and calls it in all three cases, logging why. Option A remains the fallback only when nothing at all could be scraped from the screen.

Mapping an answer onto the screen

The key gives answers as option *text*. Sphere shuffles option order on every render, so position is meaningless and text is all there is. Three helpers do the work:

Helper	Line	What it contributes
`stripHtml`	1177	The key's option text is HTML; the screen's is plain. This levels them.
`normalizeText`	1183	Collapses case, spacing and punctuation, which differ between the JSON and the rendered DOM.
`findBestOptionMatch`	1194	Fuzzy-matches one key option against the list on screen and returns its index, or `-1`.

Anything the key claims but the screen does not show is **reported rather than silently dropped** — the log prints the JSON's text verbatim, so a mismatch between the published artifact and the rendered quiz is visible instead of becoming a mystery wrong answer.

Radio, checkbox, and the field that settles it

A radio question takes exactly one answer; a checkbox question needs **every** correct option ticked or the mark is lost. Usually the DOM makes the type obvious. When it does not, the script trusts the artifact's own `multiSelection` field:

```
// the artifact says whether this is select-all-that-apply
if (scraped.inputType === 'unknown' && match.multiSelection) questionType = 'checkbox';
```

The type comment at lines 102–103 calls this "the AUTHORITATIVE answer to how many options should I tick — far more reliable than guessing from the on-screen control type", and that is right: the control can render ambiguously, but the content model cannot.

8 The answer-key plumbing

Five small functions and two Maps. Together they are the reason the AI is usually never called.

The interceptor

Sphere prefetches a JSON file for every assessment in a course, in bulk, as soon as the course opens. `setupQuizKeyInterceptor()` (line 282) attaches a `page.on('response')` listener that watches for them:

```
// only artifact JSON, of either naming convention
if (!/\/content\/[^/]+\/artifact\/.*_(quiz|assessment)\.json/i.test(url)) return;
if (!res.ok()) return;
// file it under its own resource id, but only if it actually holds questions
if (Array.isArray(json?.questions) && json.questions.length > 0) capturedKeys.set(resourceId, json);
```

Two design points are worth drawing out. It is a **Map keyed by resource id, not a single slot** — a course has many assessments and they arrive together, so each has to be filed under its own id or they overwrite one another. And the whole body is wrapped in a `try` that swallows parse errors silently: this is an opportunistic capture running inside a response handler, and a malformed body must not break the page.

> **This is why Step 2 comes before Step 3**
> The interceptor is attached *before* the course is opened. Attach it afterwards and the prefetch has already happened — the responses are gone, and every assessment falls through to the AI. It is one line in the test body and the comment beside it says only "Order matters", which is exactly right.

One regex, two jobs

`extractResourceId()` (line 272) pulls a `do_…` id out of a URL:

```
url.match(/(?:viewer\/\w+\/|content\/)(do_\w+)/)
```

The `\w+` in `viewer/\w+/` is doing quiet work. Matching *every* viewer type rather than just `quiz` is what lets `openModule()` confirm it landed on a video or a PDF, not only on an assessment — the same function serves both the answer-key lookup and the navigation check.

Going around the stripped copy

Graded assessments are served to the browser with their `isCorrect` flags removed. The published artifact on S3 still has them, and `fetchAnswerKeyFromArtifact()` (line 334) goes and gets it:

```
GET /apis/proxies/v8/action/content/v3/hierarchy/<do_id>?hierarchyType=detail
  → result.content.artifactUrl → { timeLimit, isAssessment, questions[] }
```

It runs **inside the page** rather than from Node, so the session cookies ride along without the script having to reconstruct them. Results are cached in `hierarchyKeys` — and that cache stores `null` as well as keys, which is why the code checks `.has()` rather than reading the value: an assessment with no artifact should be looked up once, not once per re-attempt.

The poison rule

`isAnswerKeyPoisoned()` (line 414) is four lines of logic with one judgement call in it:

```
const poisoned = key.questions.filter((q) ⇒ q.options.every((o) ⇒ o.isCorrect ⩵ false)).length;
return poisoned / total ⩾ 0.8;
```

Choice	Why
Count questions where **every** option is false	That is the signature of a stripped key: the questions and options survive, only the flags are gone.
80%, not 100%	A threshold rather than "all", so a handful of genuinely-unflagged questions does not throw away an otherwise perfectly good key. The same rule is used in the competency suite.
An **empty** key counts as poisoned	A key with no questions is exactly as useless as one with no answers, and treating them alike means the caller has one condition to check rather than two.

Keeping the questions even when the answers are gone

The subtlest part of `handleAssessment()` is what it does with a key it has just rejected. A poisoned key is useless for answering but still holds every question and option — so it is kept anyway, under a different name:

```
const answerKey = usable(artifactKey) ?? usable(interceptedKey);
// even a stripped key still knows the questions — keep the richer of the two
const capturedKey = answerKey ?? (artifactKey.questions.length ≥ interceptedKey.questions.length
  ? artifactKey : interceptedKey);
```

`capturedKey` is what makes Tier 2 possible: it supplies the question count, and it supplies the text of every question and option for the single batched request to Gemini. Without it the script would be back to scraping questions one screen at a time.

9 The Gemini fallback

Five functions and two pieces of state, all of them shaped by one constraint: the free tier allows **20 requests per day per model**.

That number is what dictates the whole design. A single twenty-question assessment answered one question at a time would exhaust the day's quota by itself. So when a key is poisoned — the common case — the entire assessment goes out in **one** request, built from the key's own question and option text.

The call

`callGemini()` (line 1815) makes one REST call and **returns a result object rather than throwing**, because a network failure here is a normal outcome, not an exception. Three details in the request body:

Setting	Why
`temperature: 0`	Deterministic. The same assessment should produce the same answers on a re-run.
`topP: 1`	Keeps the full distribution available, so temperature 0 picks the genuinely most likely token rather than one from a truncated set.
`thinkingBudget: 0`	Applied to the main flash model only. The reasoning in the source is exact: *"These are recall questions, not reasoning puzzles, so the budget is wasted latency."*

Telling the two kinds of 429 apart

A rate-limit response can mean two very different things, and the script distinguishes them from the error body:

Per-minute — worth waiting out

- The API usually reports the delay; the script parses it from either `retry in Ns` or `"retryDelay": "Ns"`
- No delay given? Wait `GEMINI_RETRY_WAIT_MS` (20 s)
- Capped at `GEMINI_MAX_RETRY_WAIT_MS` (65 s) so a long suggested delay cannot stall the run

Per-day — waiting cannot help

- Detected by `/PerDay/i` in the quota id
- Sets the module-level flag `geminiDailyExhausted` (line 1861)
- Every later request is skipped outright, with a plain message — rather than the run spending minutes discovering the same thing repeatedly

Model fallback and throttle

`geminiAsk()` (line 1867) wraps the call with the retry policy and a model fallback: `gemini-2.5-flash` first, `gemini-2.5-flash-lite` when the primary is rate-limited. Because the two models have separate quotas, that fallback effectively doubles the day's budget. In front of all of it, `throttleGemini()` (line 1803) simply sleeps so that no two calls are closer than 6.5 seconds — sized for 10 requests per minute, and skipped on the very first call.

The cache is what makes re-attempts free

`geminiAnswerCache` (line 977) maps a resource id to a map of *normalized question stem → correct option texts*. Both halves of that shape matter:

1. **Keyed by resource id**, so a second attempt at the same assessment reuses the answers instead of spending quota again. The log says so explicitly: `♻ Reusing cached Gemini answers … (no extra quota)`.
2. **Values are option TEXTS, not letters.** Sphere shuffles the options between attempts, so a cached "B" would be wrong the second time round. Texts survive the shuffle and are re-matched onto the screen by the same fuzzy matcher Tier 1 uses.

The honest position on using an AI inside a test

It is a fallback for a deliberate obstacle, not a shortcut. On a normal published course the answer key is intact and Gemini is never called; the log makes which path was taken unambiguous. Where it *is* called, the run's correctness depends on a third-party model being right about clinical multiple-choice questions — which is worth knowing when reading a green result. Section 11 puts that in the list of things green does not prove.

10 Completion, certificate, report

The loop ends. Now the script has to decide whether the course is actually finished — and it asks the question five different ways.

Five signals, any one of which counts

```
const completionMarker =
  pageClosed ||
  sawPopups ||
  allModulesComplete ||
  progressPct === 100 ||
  (await isCourseComplete(page).catch(() => false));
```

Signal	Where it comes from
`pageClosed`	The completion popup sometimes closes the tab outright. A dead page is therefore taken as success rather than as a crash.
`sawPopups`	`handleCompletionPopups()` found the Congratulations message or the Ratings prompt.
`allModulesComplete`	The loop exited because every sidebar node showed a check.
`progressPct === 100`	`readProgress()` read exactly 100 off the TOC progress bar.
`isCourseComplete()`	One of fourteen completion wordings is visible on screen.

The source calls this generosity deliberate: *"the app surfaces completion differently by course type."* Some courses show a popup, some just fill the progress bar, some close the tab. Requiring one specific signal would fail courses that had genuinely completed.

Fourteen ways to say "you finished"

`isCourseComplete()` (line 2205) tries fourteen regexes, in English and Hindi, each with a 300 ms timeout — it runs on *every* loop iteration, so it has to stay cheap. Two of them show the care that went into the list:

Pattern	The detail
`/congratulations[\s!,]/i`	The trailing character class is deliberate — it stops the word matching inside a longer sentence in a module description.
`/your\s+certificate/i`	Matches "Your certificate is ready" but **not** "Earn a certificate", which appears on every course page before you start.
`/rate\s+(this|the)\s+course/i`	A rating prompt only ever appears after completion, which makes it a reliable marker even though it says nothing about completion directly.
`/बधाई/i`	Hindi. The platform is bilingual and so is the completion screen.

> **It closes the ratings popup instead of using it**
> `handleCompletionPopups()` reads the popup, logs every button it offers, and then dismisses it with the X — with a comment that says why: *"the run must not leave a fake review on a real course."* Submitting a rating would be the easier path and would arguably exercise more of the product. Not doing it is the right call, and it is the kind of decision worth making explicitly rather than by omission.

The certificate, and when it is not attempted

Step 7 has two gates, and both must pass: the course must have completed, **and** a certificate must have been detected back at Step 3b. Failing either just prints a line.

State	What happens
Not complete	Skipped with `⏭ needs Congratulations/Ratings first` — the control will not be there, so trying would only add noise.
Complete, certificate attached	`generateCertificate()` runs in its own tab. Up to three attempts; the first uses the TOC already open, retries re-search the course.
Complete, no certificate	A single informational line. Not every course has one.

The generation flow itself is the fiddliest code in the file. The dialog draws the certificate onto a `<canvas>` before the download becomes available, so the script waits for that canvas, then registers the download listener and clicks **at the same time** via `Promise.all`, with a 25-second window because generating and streaming a certificate is genuinely slow. If the app answers with "certificate is taking time to generate" — one of six wordings it recognises — it closes the tab, waits twenty seconds, and starts the whole attempt again.

Downloads are saved as `test-reports/certificates/Certificate_<CourseSlug>.<ext>`, with `jfif` and `jpeg` both normalised to `jpg`. A failure anywhere in Step 7 is caught and logged — **never thrown**, because the course still completed and that is what the test is about.

The report

Step 8 writes `test-reports/course-reports/CourseReport_<slug>_<timestamp>.html` through `generateCourseReport()`. It carries the wall time, the module types in order, every warning the loop collected, the assessment table, the completion verdict, the certificate outcome and the final URL. Like Step 7 it is wrapped in a `try`: a report-writing failure must not fail a run that actually passed.

11 What green actually proves

One `expect()` in 3,195 lines. It is worth being precise about what it does and does not cover.

```
expect(
  completionMarker,
  `Course did not reach the Congratulations/Ratings completion popup (touched ${log.length} modules).
   See the course report for where it stopped.`,
).toBe(true);
```

The failure message is doing real work. It names **how many modules were touched** and points at the report, so a red run can be diagnosed without re-running an hour-long test — which matters more here than in any other spec in the suite.

Green means this

- The course was found by name and opened
- Start or Resume worked
- Modules were opened in the platform's own order, respecting gating
- Videos played to their `ended` event
- PDFs were paged to the last page
- **Every graded assessment was passed** — a failed one never marks its module complete
- The platform reached a completion state by at least one of five signals

Green does *not* mean this

- That the certificate downloaded — Step 7 never throws
- That the report was written — Step 8 never throws
- That every answer was *correct*, only that the pass mark was met
- That AI-sourced answers were right, where Tier 2 or 3 was used
- That no module was skipped after exhausting its attempts — that is a warning, not a failure
- That the run was quick, or that a learner would have had a good time

A closed page counts as success

`pageClosed` is the first of the five signals, and it is `true` whenever the tab is gone. That is right for the common case — the completion popup really does close the tab on some courses — but it means **a browser crash at the wrong moment reads as a completed course**. Nothing distinguishes "the popup closed us" from "the renderer died".

It is a narrow window and the other four signals usually corroborate, but if this run is being used as a release gate it is worth tightening: requiring `pageClosed` to be accompanied by `allModulesComplete`, or by a progress read taken before the tab went away, would close it without weakening the generosity that the five-signal design is there to provide.

How this compares with the rest of the suite

Suite	Tests	Assertions	What a green run is evidence of
Course Completion this one	1	1	The product worked end to end for one learner on one course. Broad, deep, and slow.
Course TOC	21	15 + 3	Every control on one page renders and responds. Narrow, precise, and fast.
Public Page	64	42	The marketing surface renders in two languages.
My Courses	49	17 + 1	The learner's home page renders whatever that account happens to have.

The two ends of that table are complements, not competitors. The element sweeps tell you *what* broke; this one tells you *whether a learner could finish*. A suite with only the first kind can be entirely green while the

product is unusable, and a suite with only this one can tell you something is wrong without telling you where.

12 Artifacts, run variants, failure modes

A run of this spec produces more than a pass or a fail: it produces a record of what a learner would have experienced.

What a run leaves behind

Artifact	Contents
`test-reports/course-reports/CourseReport_<slug>_<ts>.html`	**The one to open first.** Wall time, the module types in order, every warning the loop collected, the assessment table with question counts, the completion verdict, the certificate outcome and the final URL.
`test-reports/certificates/Certificate_<CourseSlug>.<ext>`	The generated certificate, if one was attached and generation succeeded.
`brand-reports/`	The branded HTML report, written on every run by the shared reporter.
`test-reports/run-reports/`	The full run report, HTML and PDF, carrying every step and every console line.
Screenshots · video · trace	`screenshot: 'on'` for every test, video retained on failure, trace on first retry.

Reading the console

This is the one spec where the log matters more than the result line, because the result line is a single boolean. The markers worth grepping for:

Mark	Line	What it tells you
🔑	Captured answer key for do_… (13 questions)	`The interceptor is working. One per assessment in the course, printed early, before any module opens.`
📝	ASSESSMENT — using answer key from the published artifact	`Which tier this assessment took. If every one of these says "published artifact", the AI was never called.`
🟩	Step 7 — 4/11 modules done	`The progress banner, one per loop iteration. The fastest way to see where a run stalled.`
⚠️	No new completion (4/11) — "…" attempt 1/2	`A module opened but did not register. The module is named, so this points straight at the defect.`
🔒	All remaining locked — nudging to unlock (retry 2/4)	`Gating did not release. Four of these in a row ends the run.`
🛑	These modules would not register as complete…	`The run gave up, and this line names every module it gave up on.`
♻️	Reusing cached Gemini answers (no extra quota)	`A re-attempt cost nothing. Good news, and confirmation the cache is working.`

Run variants

Command	Use it when
`npm run run:course`	Normal use. Prompts for account and course.
`npm run courses:list`	You need a course name that will actually match.
`npm run run:course-fresh`	Testing a *first* completion. Mints a new account, then completes as that user — the only variant that exercises enrolment and onboarding for real.
`npm run report:course`	Headless, one fixed course, blob report. The CI form.
`VERBOSE_CHECKBOX=1 …`	A multi-select question will not tick and you need to see which of the click strategies is failing.

Failure modes

Symptom	Cause	Fix
`No course selected`, immediately	`COURSE_NAME` is unset. Unlike the TOC spec this throws rather than skipping.	Use `npm run run:course`, or set it. The error text names both routes.
`Could not find course "…"`	The title did not match anything on this environment.	`npm run courses:list`. The matcher compares whole result titles, not prefixes, precisely so that a near-miss fails loudly rather than opening the wrong course.
No 🔑 lines at all, and every assessment uses Gemini	The interceptor was attached after the prefetch, or the artifact URL pattern changed.	Confirm Step 2 still runs before Step 3. If the URLs have changed, the regex at line 287 is the single place to update.
An assessment loops, re-attempting and failing	Answers are wrong. Either the key mapped badly onto the screen, or the AI answered incorrectly.	The log prints every option lettered and every pick — compare them by eye. Watch for the `⚠️ N/M not found on screen` line, which means the key's text no longer matches the rendered options.
`🛑 These modules would not register`	Those modules genuinely did not complete after two attempts each.	The most valuable failure this spec produces. Open the named module by hand and watch whether its progress call fires.
`🔒 Remaining modules stay LOCKED`	Gating never released after four nudges.	Usually a genuine prerequisite that never registered — check the modules *before* the locked ones, not the locked ones themselves.
Gemini stops answering mid-run	The per-day quota is gone. `geminiDailyExhausted` is set and later requests are skipped rather than retried.	Wait for the quota to reset, or run against a course whose key is not poisoned. The model fallback already gives two separate daily budgets.
The run hits the 60-minute timeout	Usually a module that opens but never completes, spending its attempts slowly.	Read the 🟩🟦 banners: the last one names the step and the done count, which brackets the problem to one module.
Certificate never appears	Generation is genuinely slow, or the course has none.	Not a failure — Step 7 only logs. The summary line distinguishes "attached but generation not confirmed" from "none".
A video module warns `load timeout`	The metadata never loaded within 60 s.	Recorded as a report warning, not a failure. Repeated across runs it is a CDN or encoding problem worth raising.

> **Where to start when a run goes red**
> Open the course report first, not the Playwright report. It carries the warnings, the module timeline and the assessment table in one page — and the assertion message points you there by design, because re-running to investigate costs an hour.